



Structured and hybrid querying with LangChain



Deepali Srivastava

Author

[linkedin.com/in/deepali-srivastava/](https://www.linkedin.com/in/deepali-srivastava/)



Querying structured data in LangChain



What is structured data?



Organized in a fixed format



Relational databases are common examples, queried using SQL



CSV and Excel files can also be treated as structured data



APIs provide structured data in JSON format



What is unstructured data?

No fixed format, consists of free text

Examples include documents, emails, PDFs, and transcripts

Requires understanding meaning, not exact matching

Queried using semantic search techniques



How LangChain queries structured data

Users ask questions in natural language instead of writing SQL

System converts the questions to SQL, executes them, and returns the answers



Natural language to SQL workflow



Natural language to SQL example

What is the total
revenue for 2025?



```
SELECT SUM(revenue)
FROM sales
WHERE year = 2025;
```



The total revenue for
2025 is 12500.



Querying structured and unstructured data



Structured querying (SQL)

Precise, schema-based queries on tabular data

“What are the total sales in the West region?”



Unstructured querying (document retrieval)

Meaning-based queries on text data

“Summarize customer complaints in the West region.”



Hybrid querying (combined approach)

Requires both structured and unstructured data

“What are total sales in the West region, and summarize customer complaints?”



Demo

Creating a sample database for querying



Natural language to SQL using LCEL and agents



Demo

Building a natural language-to-SQL pipeline using LCEL



Demo

Using tool-based agents for autonomous SQL reasoning



Controlled vs. autonomous approaches

LCEL is controlled
and step-by-step

Agents are flexible
and autonomous

SQLDatabaseChain
is deprecated



Challenges in querying structured data with LLMs



Ambiguous queries

Queries can be unclear

Example: “Which region is best?”

Can lead to incorrect SQL generation

Ask clarifying questions

Improve prompts for better interpretation

Guide user input with specific details



Schema understanding challenge



LLM needs to understand schema (tables and columns)



Large schemas are difficult to fully provide



Poor documentation can cause confusion



Column names may not match user queries



Can lead to incorrect SQL or invalid columns



Improving schema understanding

Schema alone is not enough

Add column descriptions
and examples

Provide only relevant tables

Improve schema context
for accuracy



SQL safety and validation

Risk of unsafe or
malicious SQL queries

Do not directly trust
LLM-generated SQL

Add validation and
only allow safe queries



Demo

Validating SQL before execution in LCEL pipelines



Demo

Ensuring query safety in agent-based systems using tool wrapping



Reliable querying with LLMs



Ensuring query correctness



Safe queries may still be incorrect



Manually validate them using rules



Apply LLM-based query checking



Demo

LCEL pipeline with SQL checker



Result verification after execution

A correct query does not always mean useful results

Results may be empty or missing

Large results are challenging to interpret

Add a result verification step



Demo

Adding result verification to the pipeline



Improving query reliability

Aa

Case sensitivity

Ensure consistent matching using functions like LOWER



Partial matches

Support flexible search using conditions like LIKE



Large result sets

Control output size using filters or LIMIT



Hybrid systems: Combining structured and unstructured data



Introduction to hybrid systems

Real-world questions require multiple data sources

Example: “Top-selling product and customer reviews about it”

Hybrid systems combine structured and unstructured data

Route query and combine results into a final answer



Demo

Controlled hybrid workflows using LCEL



Demo

Agent-based hybrid systems using `create_agent()`



Selecting the appropriate data access approach



Why strategy selection matters



Accuracy issues: wrong answers, hallucinations



Performance issues: slow queries, unnecessary computation



Cost issues: embeddings for simple queries, heavy infrastructure



Trust issues: confident but wrong answers



Understanding the query and data type

Identify whether the data is structured or unstructured

Identify whether the query needs precise results or insights

Use SQL for structured data, document retrieval for unstructured data, or a hybrid of both



Real-world complexity



Most real-world systems are hybrid



Data is spread across multiple sources



Queries are often mixed and complex



Example: “Why did sales drop in Q2?”



Hybrid combines numbers and insights



Trade-offs in system design

SQL is precise and quick but lacks context

Document retrieval understands text but is less precise

Hybrid systems are more complete but more complex

Balance precision and understanding

Choose the simplest approach





Thank you for your time!

[linkedin.com/in/deepali-srivastava/](https://www.linkedin.com/in/deepali-srivastava/)